



AN IMS [ISO 21001, ISO 9001, ISO 14001, ISO 45001] CERTIFIED TRAINING INSTITUTION

## **NETTUR TECHNICAL TRAINING FOUNDATION**

### **PROJECT REPORT**

On

## **CODE TO ARCHITECTURE VISUALIZER**

PROJECT WORK DONE BY

|                   |            |
|-------------------|------------|
| RANJAN BABU V     | NEC0823004 |
| SHWETHA A         | NEC0823029 |
| AKLIMA BEGUM SARA | NEC0823115 |
| GOUTHAM S         | NEC0823105 |
| YOGESH S          | NEC0823014 |

**DIPLOMA IN  
COMPUTER ENGINEERING & IT INFRASTRUCTURE,  
NTTF ELECTRONICS CENTRE  
ELECTRONICS CITY, BANGALORE-560100  
2023-2026**



AN IMS [ISO 21001, ISO 9001, ISO 14001, ISO 45001] CERTIFIED TRAINING INSTITUTION

## NETTUR TECHNICAL TRAINING FOUNDATION

### CERTIFICATE

This is to certify that the project titled.

## CODE TO ARCHITECTURE VISUALIZER

is a Bonafede report of project work done by

|                   |            |
|-------------------|------------|
| RANJAN BABU V     | NEC0823004 |
| SHWETHA A         | NEC0823029 |
| AKLIMA BEGUM SARA | NEC0823115 |
| GOUTHAM S         | NEC0823105 |
| YOGESH S          | NEC0823014 |

IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF  
DIPLOMA IN COMPUTER ENGINEERING & IT INFRASTRUCTURE UNDER THE  
INSTITUTION NETTUR TECHNICAL TRAINING FOUNDATION, ELECTRONICS  
CENTRE, BANGALORE DURING THE ACADEMIC YEAR 2023-2026

PROJECT GUIDE

PROJECT COORDINATOR

COURSE HEAD

INTERNAL EXAMINER

EXTERNAL EXAMINER

PRINCIPAL

## ACKNOWLEDGEMENT

We would like to express our deepest gratitude and appreciation to all those who have supported us throughout the completion of this final year project.

To begin with, we would like to thank Principal **Dr. Somanathan K, NTTF NEC**, for his consistent encouragement and support which pushed us to do better. We are thankful to our Course Head of CP08 **Mr. Loganathan B** and Program Manager **Mrs. Preetha S** for being a source of inspiration throughout our project.

We would also like to extend our sincere thanks to our project coordinators **Mrs. Preetha S** for their invaluable guidance, constant encouragement, and constructive feedback. Their expertise and mentorship have played a crucial role in shaping this project and enhancing its quality. We are indebted and grateful to **Mr. Loganathan B** our project guide for his valuable guidance for each and every stage of our project as reality.

In conclusion, this project would not have been possible without the collective efforts and support of all those mentioned above and the team members of the project. We are truly grateful for their contributions, and we feel fortunate to have been surrounded by such incredible individuals during this project.

## ABSTRACT

The **Code-to-Visualizer System** is an intelligent web-based application designed to simplify code understanding by converting source code into visual flowcharts and structured explanations. The system allows users to write or upload Python and JavaScript programs, execute them securely, and automatically generate flowchart representations that illustrate the program logic in an easy-to-understand manner. This helps students, beginners, developers, and educators analyze complex code structures more efficiently.

The system includes secure user authentication with OTP verification, encrypted code storage, password hashing, and admin monitoring features to ensure data security and controlled access. Additional functionalities such as code execution, downloadable flowcharts, recent activity restoration, task-based learning modules, and performance tracking improve the overall learning and development experience. The application also provides an admin dashboard for monitoring users, submissions, reports, and system activities.

The main objective of the project is to reduce the difficulty of understanding programming logic by transforming textual source code into interactive visual representations. By integrating code execution, visualization, and security features into a single platform, the system enhances programming education, debugging, and logical analysis in an efficient and user-friendly manner.

| SLNO.                                               | CONTENT                                       | PAGE NO |
|-----------------------------------------------------|-----------------------------------------------|---------|
| <b>CHAPTER - I INTRODUCTION</b>                     |                                               |         |
| 1.1                                                 | Introduction.....                             |         |
| 1.2                                                 | Objective of the Project.....                 |         |
| 1.3                                                 | System Study.....                             |         |
| 1.3.1                                               | Study on Existing System.....                 |         |
| 1.3.2                                               | Demerits of Existing System.....              |         |
| 1.3.3                                               | Study on Proposed System.....                 |         |
| 1.3.4                                               | Merits of Proposed System.....                |         |
| 1.3.5                                               | Literature Survey .....                       |         |
| <b>CHAPTER – II SYSTEM ANALYSIS</b>                 |                                               |         |
| 2.1                                                 | User Requirement Specifications.....          |         |
| 2.2                                                 | Software Requirement Specification (SRS)..... |         |
| 2.3                                                 | System Specifications.....                    |         |
| 2.3.1                                               | Hardware Specifications.....                  |         |
| 2.3.2                                               | Software Specifications.....                  |         |
| 2.4                                                 | Final Outline of Proposed System.....         |         |
| <b>CHAPTER – III DESIGN AND DEVELOPMENT PROCESS</b> |                                               |         |
| 3.1                                                 | Fundamental Design Concepts.....              |         |
| 3.1.1                                               | Modules and Functionalities.....              |         |
| 3.1.2                                               | Description of Modules .....                  |         |
| 3.2                                                 | Design Notations.....                         |         |
| 3.2.1                                               | DFD Symbols.....                              |         |
| 3.2.2                                               | Data Flow Diagram.....                        |         |
| 3.2.3                                               | ER Diagram.....                               |         |
| 3.3                                                 | Design Process.....                           |         |
| 3.3.1                                               | Input Design.....                             |         |
| 3.3.2                                               | Output Design.....                            |         |
| 3.3.3                                               | Database Design.....                          |         |

|       |                                   |
|-------|-----------------------------------|
| 3.4   | Development Approach.....         |
| 3.4.1 | Development Model / Approach..... |

## **CHAPTER – IV PLANNING**

|     |                  |
|-----|------------------|
| 4.1 | Gantt Chart..... |
|-----|------------------|

## **CHAPTER – V TESTING AND IMPLEMENTATION**

|       |                                |
|-------|--------------------------------|
| 5.1   | Testing.....                   |
| 5.1.1 | Testing and Methodologies..... |
| 5.1.2 | Test Cases.....                |
| 5.2   | System Implementation.....     |
| 5.2.1 | Implementation Procedures..... |

## **CHAPTER – VI ADVANTAGES AND DISADVANTAGES**

|     |                                    |
|-----|------------------------------------|
| 6.1 | Advantages and Disadvantages ..... |
|-----|------------------------------------|

## **CHAPTER – VII CONCLUSION**

|     |                                    |
|-----|------------------------------------|
| 7.1 | Conclusion.....                    |
| 7.2 | Scope for Future Enhancements..... |

## **ANNEXURE - A**

|                  |
|------------------|
| Sample Code..... |
|------------------|

## **ANNEXURE - B Abbreviation**

.....

## **BIBLIOGRAPHY.....**

# **CHAPTER - I**

## **INTRODUCTION**

## 1.1 INTRODUCTION

In today's software-driven world, understanding how a program flows and behaves is a fundamental challenge faced by students, developers, and educators alike. Reading raw source code — especially for beginners — is a cognitively demanding task that often leads to confusion, errors, and slow learning. Traditional debugging tools and textbooks provide limited visual context, leaving learners to mentally simulate program execution on their own.

Ragsy (Code Architecture Visualizer) is a full-stack web application designed to bridge this gap. It transforms Python and JavaScript source code into interactive, step-by-step flowcharts using Abstract Syntax Tree (AST) analysis and graph-based rendering. Users paste their code into a browser-based editor, and the platform instantly produces a structured, colour-coded flowchart that visually represents the control flow, function calls, loops, conditionals, and return statements of the program.

Beyond flowchart generation, Ragsy provides a live code execution environment where users can run their code directly in the browser, observe real-time output, and provide input values interactively. The platform supports both Python Basics (executed via a sandboxed server-side Python runtime) and JavaScript Basics (executed via Node.js). All code submissions are securely stored in an AES-256 encrypted database, and users can retrieve their recent activity to restore and re-analyse past work. Ragsy is built with a modern technology stack: React (Vite) on the frontend, FastAPI (Python) on the backend, and Microsoft SQL Server as the development database (PostgreSQL for production deployment). The platform features secure multi-user authentication with SHA-256

password hashing, SMS-based OTP via Fast2SMS Phone Authentication for password recovery, and a dedicated Admin Dashboard for platform management and oversight.

## **1.2 OBJECTIVE OF OUR PROJECT**

1. To provide an interactive visual representation of Python and JavaScript source code as flowcharts, helping learners and developers understand program logic without deep manual analysis.
2. To offer a live, sandboxed code execution environment supporting both Python and JavaScript directly in the browser.
3. To implement a secure, multi-user authentication system with SHA-256 hashed passwords, SMS-based OTP password recovery via Fast2SMS, and 24-hour session expiry.
4. To encrypt all user-submitted code in the database using AES-256 (Fernet), ensuring complete user data privacy and platform trustworthiness.
5. To build an Admin Dashboard that allows platform administrators to monitor users, view submission statistics, and manage the system without direct database access.
6. To support high-quality downloadable flowchart exports as PNG images, capturing all node connections and SVG edges using html-to-image.
7. To create a clean, responsive, and humanised user interface accessible to students and beginner programmers without any installation.
8. To design the system for free cloud deployment using open-source tools (Vercel, Railway, Supabase, Fast2SMS).

## **1.3 System Study**

### **1.3.1 Study on Existing System**

Several tools currently exist for visualising code or understanding program flow:

1. **Python Tutor ([pythontutor.com](https://pythontutor.com))**: Animates Python code execution step by step. Useful for teaching but does not produce flowcharts and supports Python only.
2. **VS Code Extensions** : IDE-based plugins generating basic flowcharts. Require installing a development environment — inaccessible to beginners.
3. **Lucidchart and Draw.io**: Manual diagramming tools where users hand-draw flowcharts. Time-consuming and require prior knowledge of flowchart conventions.
4. **Code2Flow**: Generates flowcharts from pseudocode, not actual source code. Limited language support and no interactive execution environment.
5. **Mermaid.js**: A text-based diagram tool requiring users to write diagram syntax, not actual code. Not suitable for direct code visualisation.

### **1.3.1.1 Demerits of Existing System**

1. No existing platform combines live code execution with real-time flowchart generation in a single interface.
2. Most tools are desktop-based or require installation, making them inaccessible to students without technical setup knowledge.
3. Existing tools do not support both Python and JavaScript in the same platform.
4. None of the existing tools offer encrypted code storage, multi-user accounts, SMS-based password recovery, or an Admin Dashboard.
5. Flowchart export quality in existing tools is often poor, with edges and connections not properly captured in exported images.
6. No existing tool is designed specifically for beginner programmers learning basic programming concepts interactively.

### **1.3.2 Study on Proposed System**

Ragsy addresses the shortcomings of existing systems by providing a unified, web-based platform that combines code visualisation, execution, and secure data management. The proposed system is fully browser-based (no installation required), language-aware (Python Basics and JavaScript Basics), secure by design (hashed passwords, encrypted submissions, session expiry), and admin-controlled (dedicated dashboard for platform oversight).

### **1.3.2.1 Merits of Proposed System**

1. Real-time flowchart generation from actual source code using AST analysis, with Dagre automatic layout ensuring non-overlapping node placement.
2. Live code execution with input/output support directly in the browser for both Python and JavaScript.
3. High-quality PNG export of flowcharts including all edge connections and arrows, rendered using html-to-image.
4. Secure multi-user authentication with SHA-256 hashed passwords and Fast2SMS SMS OTP-based password recovery.
5. AES-256 (Fernet) encryption of all code submissions stored in the database — only the submitting user can retrieve their code.
6. Admin Dashboard with real-time platform statistics, user management (view/delete), and encrypted submission monitoring.
7. Recent Activity panel allowing users to restore and re-analyse previously submitted code.
8. Humanised, responsive UI built with React for an approachable, accessible learning experience.
9. Designed for zero-cost cloud deployment: Vercel (frontend), Railway (backend), Supabase (PostgreSQL), Fast2SMS (SMS OTP).

## **CHAPTER – II**

### **SYSTEM ANALYSIS**

## **2.1 User Requirement Specifications (URS)**

The system shall allow users to register using a username, mobile number, and password. Registered users shall be able to log in securely and access a personalised dashboard. The application shall enable users to paste Python or JavaScript code and instantly generate a corresponding flowchart. Users shall also be able to execute code and view real-time standard output and error messages. When the program requires user input, the system shall provide an interactive method for entering input values during execution.

Additionally, the application shall allow users to download generated flowcharts as high-quality PNG images with connected edges. Users shall be able to view, restore, and re-analyse their recent code submissions for future reference. The system shall support password reset functionality through SMS OTP verification sent to the user's registered mobile number. Admin users shall have access to a separate dashboard containing user information and submission details. Furthermore, the system shall ensure that user credentials, submitted code, and other sensitive data are encrypted and securely protected to maintain privacy and security.

## **2.2 Software Requirement Specification (SRS)**

### **Functional Requirements**

1. FR-01: The system shall parse Python code using the ast module and JavaScript using a custom tokeniser to generate flowchart nodes and edges.
2. FR-02: The system shall execute Python code in a sandboxed subprocess and JavaScript via Node.js, both with a 5-second timeout.

3. FR-03: The system shall hash all passwords using SHA-256 before storage. Plain text passwords shall never be stored.
4. FR-04: The system shall encrypt all code submissions using AES-256 (Fernet) before storage, and decrypt them only for the submitting user.
5. FR-05: The system shall send a 6-digit SMS OTP via Fast2SMS Phone Auth for password reset, expiring after 5 minutes.
6. FR-06: The system shall automatically expire user sessions after 24 hours.
7. FR-07: The system shall apply Dagre auto-layout to prevent node overlapping on initial flowchart render.
8. FR-08: The Admin Dashboard shall display total users, submissions, flowcharts generated, and new users per week.
9. FR-09: Admin users shall be able to delete non-admin user accounts from the dashboard.
10. FR-10: The system shall export flowcharts as PNG images capturing all SVG edges using html-to-image.

### **Non-Functional Requirements**

1. NFR-01 Performance: Code execution shall complete within 5 seconds; flowchart generation within 3 seconds for programs up to 500 lines.
2. NFR-02 Security: No plain-text passwords or unencrypted code shall ever be stored in the database.
3. NFR-03 Usability: The platform shall be fully functional on modern browsers (Chrome, Firefox, Edge) without any client-side installation.

4. NFR-04 Scalability: The API shall be stateless and deployable on containerised infrastructure (Railway, Render, etc.).
5. NFR-05 Availability: The deployed system shall target 99% uptime using cloud-managed services.

## 2.3 System Specifications

### 2.3.1 Hardware Specifications

| Component        | Minimum Requirements          | Recommended Requirements          |
|------------------|-------------------------------|-----------------------------------|
| <b>Processor</b> | : Intel Core i3 / AMD Ryzen 3 | : Intel Core i5 / AMD Ryzen 5     |
| <b>RAM</b>       | : 4 GB                        | : 8 GB or above                   |
| <b>Storage</b>   | : 256 GB HDD                  | : 512 GB SSD                      |
| <b>Network</b>   | : Broadband 1 Mbps            | : Broadband 10 Mbps or higher     |
| <b>Display</b>   | : 1280 × 720                  | : 1920 × 1080 or higher           |
| <b>Browser</b>   | : Chrome 90+ / Firefox 88+    | : Latest Chrome or Microsoft Edge |

### 2.3.2 Software Specifications

| Category             | Technology                      |
|----------------------|---------------------------------|
| Frontend Framework   | : React via Vite                |
| Backend Framework    | : FastAPI (Python)              |
| Database (Dev)       | : Microsoft SQL Server          |
| Database (Prod)      | : PostgreSQL (Supabase)         |
| JS Execution Runtime | : Node.js                       |
| Flow Rendering       | : React Flow (@xyflow/react)    |
| Graph Auto-Layout    | : Dagre                         |
| PNG Export           | : html-to-image                 |
| SMS OTP              | : Fast2SMS Phone Auth           |
| Code Encryption      | : cryptography (Fernet/AES-256) |
| Password Hashing     | : hashlib SHA-256               |
| Deployment (FE)      | : Vercel                        |
| Deployment (BE)      | : Railway                       |
| Version Control      | : Git / GitHub                  |

## **2.4 Final Outline of Proposed System**

Ragsy is structured as a three-tier web application. The Presentation Tier is a React SPA (Vite) served from Vercel, handling all UI rendering, flowchart display via React Flow, and code editing. The Application Tier is a FastAPI REST API served from Railway, handling authentication, AST parsing, sandboxed execution, database operations, and admin functions. The Data Tier is Microsoft SQL Server in development and PostgreSQL (Supabase) in production, storing encrypted user data, code submissions, flowchart metadata, OTP records, and activity logs. All communication between tiers uses JSON over HTTPS, and no sensitive data is transmitted or stored in plain text.

## **2.5 Literature Survey**

- The Python Software Foundation Python 3.12 documentation for the AST (Abstract Syntax Tree) module provided the foundation for AST-based code parsing and the generation of flowchart nodes and edges in the Ragsy application. The React Flow Documentation was used as the primary reference for implementing interactive flowchart rendering features such as node dragging, zooming, and graphical visualization. The Dagre Graph Layout Algorithm supported the automatic hierarchical arrangement of flowchart nodes to avoid overlapping during rendering.

- The OWASP Password Storage Cheat Sheet guided the implementation of SHA-256 password hashing and session security design within the system. The Fast2SMS Phone Authentication Documentation served as the basis for implementing SMS OTP-based password recovery functionality. Additionally, FastAPI Documentation was referenced for backend development due to its asynchronous performance, automatic OpenAPI documentation support, and built-in validation features.
- Furthermore, the html-to-image GitHub Repository helped solve the SVG edge capture issue during PNG export of flowcharts, replacing the limitations of html2canvas. Finally, the Supabase Documentation provided guidance for implementing a free-tier PostgreSQL hosting strategy for production deployment of the Ragsy platform.

## **CHAPTER – III**

# **DESIGN AND DEVELOPMENT PROCESS**

## 3.1 Fundamental Design Concepts

### 3.1.1 Modules and Functionalities

| Module                 | Key Functionalities                                                                                             |
|------------------------|-----------------------------------------------------------------------------------------------------------------|
| Authentication Module  | Register, login, logout, session management, forgot password (SMS OTP), admin detection, 24-hour session expiry |
| Dashboard Module       | Language selection (Python Basics / JavaScript Basics), user greeting, navigation to editor                     |
| Code Editor Module     | Code input text area, sample code loader, language indicator, run/generate triggers                             |
| Flowchart Engine       | AST parsing, node/edge JSON generation, Dagre layout, React Flow rendering, interactive drag/zoom, PNG export   |
| Execution Engine       | Sandboxed Python subprocess, Node.js child process, input scanning modal, stdout /stderr capture, 5s timeout    |
| Explanation Module     | AI-powered plain-English explanation of submitted code structure and logic                                      |
| Recent Activity Module | Submission history display, code restore with correct language, activity log                                    |
| Admin Dashboard Module | User management (view/delete), submission monitoring, platform statistics, security status display              |
| Database Module        | SQL CRUD operations, Fernet AES-256 encrypt/decrypt, session store, OTP management                              |

### 3.1.2 Description of Modules

#### Authentication Module

Handles all user identity operations. On registration, passwords are SHA-256 hashed before storage. On login, a unique 32-character alphanumeric token is generated and stored in an in-memory session dictionary with a 24-hour expiry timestamp. The forgot password flow invokes Fast2SMS Phone Authentication to deliver an SMS OTP to the user's registered mobile number. Admin users (is\_admin=1 in USERS table) are automatically routed to the Admin Dashboard on login.

## Flowchart Engine

The core feature of Ragsy. The backend parses submitted code using Python's built-in ast module (Python) or a custom tokeniser (JavaScript), walking the syntax tree to identify control flow constructs: function definitions, if/elif/else branches, for/while loops, try/except blocks, return statements, and function calls. Each construct becomes a typed node (process, decision, terminal, I/O) in a directed graph; relationships become labelled edges. The backend returns nodes and edges as JSON. On the frontend, the Dagre library recalculates positions using a top-to-bottom hierarchical layout (TB direction, nodesep=60, ranksep=80) before React Flow renders the interactive diagram.

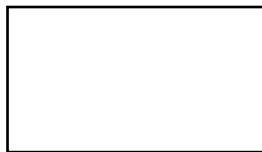
## Execution Engine

Runs user code in an isolated environment. Python code executes in a subprocess with stdout/stderr capture and a strict 5-second timeout to prevent infinite loops. Before execution, the code is scanned for `input()` calls; if detected, a modal prompts the user to provide values which are injected as simulated stdin. JavaScript code executes via a Node.js child process with an injected `input()` shim for interactive programs. The subprocess design mitigates dangerous operations by isolating execution from the server's file system and network.

### 3.2 Design Notations

#### 3.2.1 DFD symbols

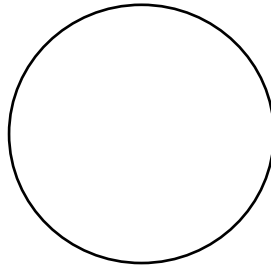
A Square defines the source/destination of system data.



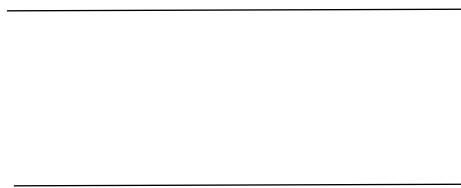
An arrow identifies data flow. It is the pipeline through which information flows.



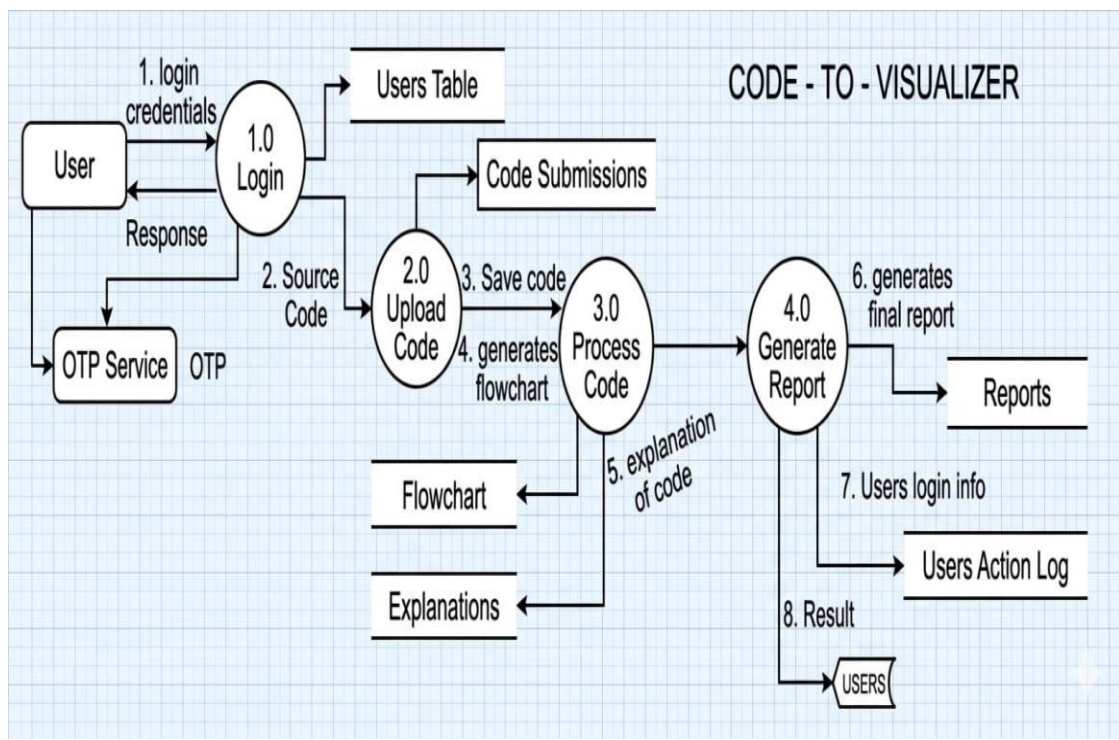
circle represents a process that transforms incoming dataflow into outgoing dataflows.



Open rectangle represents the data store or tab.



### **3.2.2 DFD FLOW DIAGRAM**



The Data Flow Diagram (DFD) of the Code-to-Visualizer System illustrates the flow of data between users, processes, and data stores within the system.

1. **Login(1.0):**

The process begins when the User enters login credentials into the system. The credentials are verified using the Users Table. An OTP Service is used to send and validate the OTP for secure authentication. After successful verification, a response is returned to the user.

2. **Upload\_Code(2.0):**

After logging in, the user uploads the source code to the system. The uploaded code is stored in the Code Submissions data store for future reference and processing.

3. **Process\_Code(3.0):**

The uploaded code is analyzed and processed by the system. During this stage, the system saves the code, generates a flowchart representation, and produces a detailed code explanation. The generated outputs are stored in the Flowchart and Explanations data stores.

4. **Generate\_Report(4.0):**

The processed results are then sent to the report generation module. This module creates the final report by combining the generated flowchart, code explanations, and processing details. The report is stored in the Reports data store.

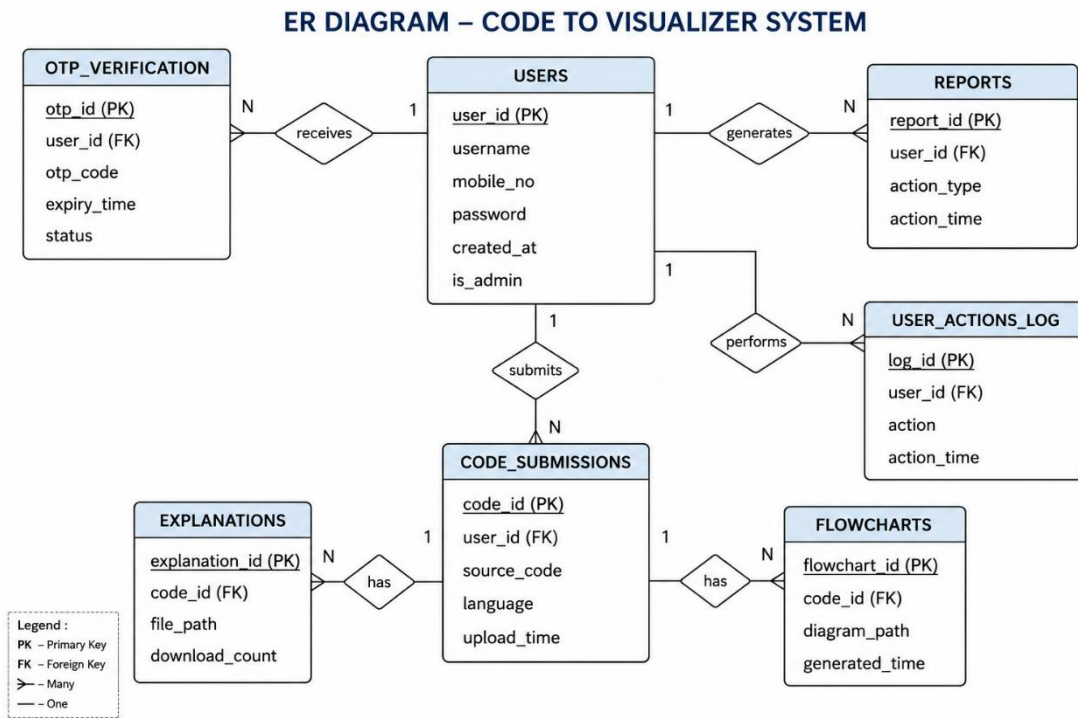
5. **User\_Activity\_Logging:**

The system records user login information and actions in the Users Action Log for monitoring, auditing, and security purposes.

6. **Result\_Delivery:**

Finally, the generated report and processed results are delivered back to the Users, allowing them to view the visualized code output, explanations, and reports.

### 3.2.3 ER DIAGRAM



## 3.3 Design Process

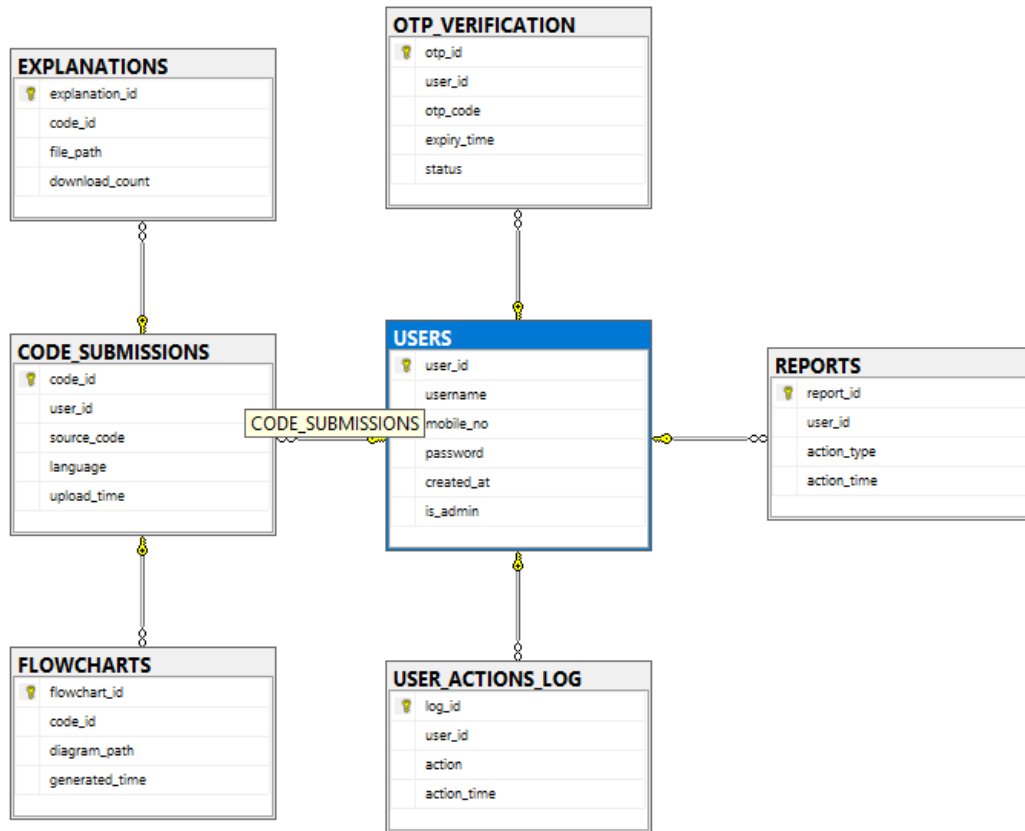
### 3.3.1 Input Design

| Input Field   | Source                         | Validation Rules                        | Processing Applied                                |
|---------------|--------------------------------|-----------------------------------------|---------------------------------------------------|
| Username      | Registration Form              | Alphanumeric, 3–30 chars, unique in DB  | Stored as plain text                              |
| Mobile Number | Registration / Forgot Password | 10-digit number with country code (+91) | Used for Fast2SMS OTP only                        |
| Password      | Registration / Login           | Minimum 6 characters                    | SHA-256 hashed before DB storage                  |
| Source Code   | Code Editor Text area          | Non-empty, max ~10,000 chars            | AST-parsed; Fernet-encrypted before DB storage    |
| User Inputs   | Input Modal (runtime)          | String values matching input() prompts  | Injected as stdin to execution subprocess         |
| OTP Code      | 6 OTP Input Boxes              | 6-digit numeric code                    | Compared with DB record; expiry timestamp checked |
| New Password  | Reset Password Form            | Minimum 6 characters                    | SHA-256 hashed; replaces existing password in DB  |

### 3.3.2 output Design

| Output            | Format                      | Destination           | Description                                                        |
|-------------------|-----------------------------|-----------------------|--------------------------------------------------------------------|
| Flowchart Diagram | Interactive SVG graph       | Browser canvas        | Colour-coded nodes and directed edges showing program control flow |
| Flowchart PNG     | PNG image (2x resolution)   | File download         | Full-quality export with all SVG edges captured by html-to-image   |
| Code Output       | Plain text (stdout/stderr)  | Console Panel         | Real-time output from sandboxed code execution                     |
| Code Explanation  | Formatted text sections     | Explanation Panel     | AI-generated plain-English description of code structure           |
| Activity History  | Card list with metadata     | Recent Activity Panel | User's last 10 submissions with language, date, and restore button |
| Admin Stats       | Stat cards and tables       | Admin Dashboard       | Platform metrics and user/submission management UI                 |
| Session Token     | 32-char alphanumeric string | Browser localStorage  | Sent as query param in all authenticated API requests              |

### 3.3.3 DATABASE DIAGRAM



## 3.4 Development Approach

### 3.4.1 Development Model / Approach

Ragsy was developed using an Agile Iterative Development model divided into four sprints, each delivering a working, testable increment of the product:

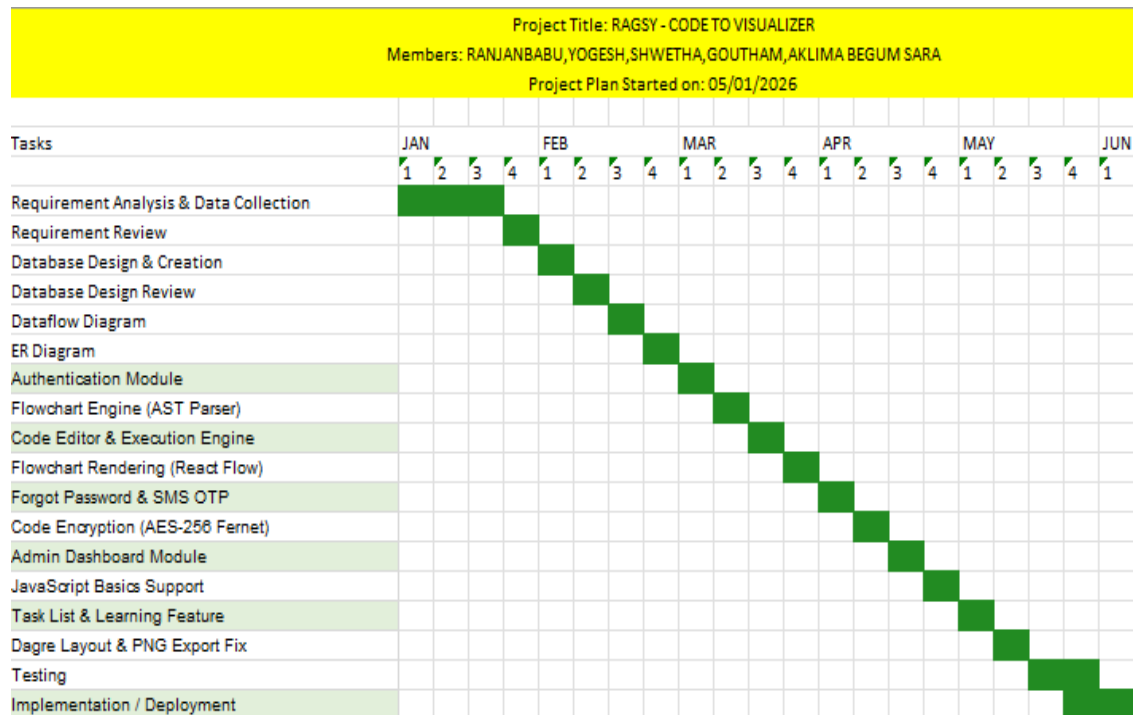
| Sprint   | Duration | Focus Area          | Key Deliverables                                                                                               |
|----------|----------|---------------------|----------------------------------------------------------------------------------------------------------------|
| Sprint 1 | Week 1–2 | Core Infrastructure | FastAPI scaffold, SQL Server schema, register/login API, React SPA with routing, basic auth UI                 |
| Sprint 2 | Week 3–4 | Core Features       | AST parser, flowchart engine, React Flow rendering, code execution engine, console output panel                |
| Sprint 3 | Week 5–6 | Security & UX       | SHA-256 hashing, Fernet encryption, Dagre layout, PNG export fix, forgot password (Firebase OTP), humanised UI |
| Sprint 4 | Week 7–8 | Admin Deployment &  | Admin Dashboard, JavaScript Basics, Recent Activity, session expiry, free cloud deployment, documentation      |

## CHAPTER – IV

### PLANNING

**S**

## 4.1 GANTT CHART



- A Gantt chart is a project management tool used to visually represent the schedule and progress of a project over a specific period of time.
- It displays the various tasks or activities involved in the project along with their start dates, end dates, and duration using horizontal bars.
- The chart helps in planning, organizing, and tracking project activities efficiently by showing the sequence of tasks and their dependencies.
- It is widely used in software development and final year projects to monitor progress, manage deadlines, allocate resources, and ensure the timely completion of the project.

## **CHAPTER – V**

# **TESTING AND IMPLEMENTATION**

## 5.1 Testing

### 5.1.1 Testing and Methodologies

**Unit Testing:** Individual backend functions including `hash_pw()`, `encrypt code()`, `decrypt code()`, `gen_otp()`, `run_code_safe()`, and `db_user_create()` were tested in isolation to verify correct behaviour, error handling, and edge cases.

**Integration Testing:** All API endpoints were tested using the FastAPI TestClient, verifying correct HTTP status codes, response schemas, and database interactions for routes including `/auth/register`, `/auth/login`, `/auth/forgot-password`, `/auth/reset-password`, `/visualize`, `/run`, `/activities`, and all `/admin/*` routes.

**Frontend Testing:** React components (`AuthPage`, `ForgotPasswordView`, `FlowDiagram`, `EditorPage`, `AdminPage`) were tested manually in the browser for state management, rendering correctness, error display, and complete user interaction flows.

**Security Testing:** Password hashing was verified by confirming SHA-256 output consistency. Fernet encryption/decryption round-trips were tested to ensure code retrieval integrity. Admin route protection was confirmed by verifying HTTP 403 responses for non-admin tokens.

**Browser Compatibility:** Tested on Google Chrome (v120+), Mozilla Firefox (v120+), and Microsoft Edge (v120+) for consistent rendering and functionality.

**Performance Testing:** Infinite loop programs were tested to verify the 5-second subprocess timeout. Flowchart generation was tested with programs up to 200 lines to verify sub-3-second response times.

## 5.12 Test Cases

| TC ID | Description                                    | Input                                                                          | Expected Result                                                          | Actual Result                                                            | Result    |
|-------|------------------------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------|-----------|
| TC-01 | User Registration — Valid Input                | Username: ranjan<br>Mobile: 9786006541<br>Password: Test@123                   | Account created successfully. Token returned. Redirected to Dashboard.   | Account created successfully. Token returned. Redirected to Dashboard.   | ✓<br>PASS |
| TC-02 | User Registration — Duplicate Username         | Username: ranjan (already exists)<br>Mobile: 9786006542<br>Password: Test@123  | HTTP 400 error: Username already taken                                   | HTTP 400 error: Username already taken                                   | ✓<br>PASS |
| TC-03 | User Registration — Duplicate Mobile Number    | Username: newuser<br>Mobile: 9786006541 (already exists)<br>Password: Test@123 | HTTP 400 error: Mobile number already registered                         | HTTP 400 error: Mobile number already registered                         | ✓<br>PASS |
| TC-04 | User Registration — Weak Password              | Username: testuser<br>Mobile: 9111111111<br>Password: 123                      | HTTP 400 error: Password must be at least 6 characters                   | HTTP 400 error: Password must be at least 6 characters                   | ✓<br>PASS |
| TC-05 | User Login — Valid Credentials                 | Username: ranjan<br>Password: Test@123                                         | Login successful. Token returned. Redirected to Dashboard.               | Login successful. Token returned. Redirected to Dashboard.               | ✓<br>PASS |
| TC-06 | User Login — Wrong Password                    | Username: ranjan<br>Password: WrongPass                                        | HTTP 401 error: Incorrect password                                       | HTTP 401 error: Incorrect password                                       | ✓<br>PASS |
| TC-07 | User Login — Non-existent Username             | Username: unknownuser<br>Password: Test@123                                    | HTTP 401 error: Username not found                                       | HTTP 401 error: Username not found                                       | ✓<br>PASS |
| TC-08 | Admin Login — Valid Admin Credentials          | Username: admin<br>Password: admin                                             | Login successful. is_admin=1. Redirected to Admin Dashboard.             | Login successful. is_admin=1. Redirected to Admin Dashboard.             | ✓<br>PASS |
| TC-09 | Admin Login — Regular User Cannot Access Admin | Username: ranjan (is_admin=0)<br>Password: Test@123                            | Login successful. Redirected to User Dashboard (not Admin).              | Login successful. Redirected to User Dashboard (not Admin).              | ✓<br>PASS |
| TC-10 | Session Expiry After 24 Hours                  | Token older than 24 hours used for API call                                    | HTTP 401 error: Session expired. Please log in again.                    | HTTP 401 error: Session expired. Please log in again.                    | ✓<br>PASS |
| TC-11 | Forgot Password — Valid Registered Mobile      | Mobile: +919786006541 (registered)                                             | OTP sent to mobile. Step 2 (OTP entry) displayed. Timer starts at 05:00. | OTP sent to mobile. Step 2 (OTP entry) displayed. Timer starts at 05:00. | ✓<br>PASS |
| TC-12 | Forgot Password —                              | Mobile: +919000000000(not registered)                                          | HTTP 404 error: No account found with this mobile number                 | HTTP 404 error: No account found with this mobile number                 | ✓<br>PASS |

|       |                                             |                                                              |                                                                                                  |                                                                                                  |           |
|-------|---------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------|
| TC-13 | Forgot Password — Correct OTP Entered       | OTP: correct 6-digit code within 5 minutes                   | OTP verified. Step 3 (new password) displayed.                                                   | OTP verified. Step 3 (new password) displayed.                                                   | ✓<br>PASS |
| TC-14 | Forgot Password — Wrong OTP Entered         | OTP: incorrect 6-digit code                                  | HTTP 400 error: Incorrect OTP. Please try again.                                                 | HTTP 400 error: Incorrect OTP. Please try again.                                                 | ✓<br>PASS |
| TC-15 | Forgot Password — Expired OTP               | Correct OTP entered after 5 minutes                          | HTTP 400 error: OTP expired. Please request a new one.                                           | HTTP 400 error: OTP expired. Please request a new one.                                           | ✓<br>PASS |
| TC-16 | Forgot Password — New Password Too Short    | New Password: 123 (less than 6 chars)                        | HTTP 400 error: Password must be at least 6 characters                                           | HTTP 400 error: Password must be at least 6 characters                                           | ✓<br>PASS |
| TC-17 | Forgot Password — Successful Password Reset | Mobile: registered<br>OTP: correct New Password: NewPass@123 | Password updated. Success message shown. Redirected to Sign In.                                  | Password updated. Success message shown. Redirected to Sign In.                                  | ✓<br>PASS |
| TC-18 | Login With New Password After Reset         | Username: ranjan<br>Password: NewPass@123 (after reset)      | Login successful. Redirected to Dashboard.                                                       | Login successful. Redirected to Dashboard.                                                       | ✓<br>PASS |
| TC-19 | Flowchart Generation — Python Hello World   | print("Hello World")                                         | Flowchart with START → process node → END generated. Dagle layout applied. No overlapping nodes. | Flowchart with START → process node → END generated. Dagle layout applied. No overlapping nodes. | ✓<br>PASS |
| TC-20 | Flowchart Generation — Python If/Else       | x = 10 if x > 5: print('big') else: print('small')           | Diamond decision node with Yes/No branches. Green Yes edge. Red No edge. Straight connections.   | Diamond decision node with Yes/No branches. Green Yes edge. Red No edge. Straight connections.   | ✓<br>PASS |
| TC-21 | Flowchart Generation — Python For Loop      | for i in range(5): print(i)                                  | Loop node with iteration edges generated. Proper back-edge shown.                                | Loop node with iteration edges generated. Proper back-edge shown.                                | ✓<br>PASS |
| TC-22 | Flowchart Generation — Python Function      | def add(a, b): return a + b<br>result = add(2, 3)            | Function definition node and function call node generated with edges.                            | Function definition node and function call node generated with edges.                            | ✓<br>PASS |
| TC-23 | Flowchart Generation — JavaScript Code      | let x = 10; if(x > 5){ console.log('big'); }                 | JavaScript flowchart generated with decision diamond. No SyntaxError.                            | JavaScript flowchart generated with decision diamond. No SyntaxError.                            | ✓<br>PASS |
| TC-24 | Flowchart Generation — Empty Code           | Empty string submitted                                       | HTTP 400 error: Code cannot be empty                                                             | HTTP 400 error: Code cannot be empty                                                             | ✓<br>PASS |

|       |                                                   |                                                                      |                                                                                           |                                                                                           |           |
|-------|---------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-----------|
| TC-25 | Flowchart Tab Auto-Switch on Generate             | Valid Python code submitted                                          | Diagram tab automatically activates when Generate Flowchart is clicked.                   | Diagram tab automatically activates when Generate Flowchart is clicked.                   | ✓<br>PASS |
| TC-26 | Flowchart Node Dragging                           | Drag a node to new position                                          | Node moves to new position. Edges follow. Other nodes unaffected.                         | Node moves to new position. Edges follow. Other nodes unaffected.                         | ✓<br>PASS |
| TC-27 | Flowchart PNG Download — With Edges               | Click Download PNG on generated flowchart                            | PNG file downloaded. All nodes visible. All edge connections and arrows visible in image. | PNG file downloaded. All nodes visible. All edge connections and arrows visible in image. | ✓<br>PASS |
| TC-28 | Python Code Execution — Basic Output              | <code>print('Hello World')</code>                                    | stdout: Hello World displayed in Output panel.                                            | stdout: Hello World displayed in Output panel.                                            | ✓<br>PASS |
| TC-29 | Python Code Execution — Arithmetic                | <code>print(2 + 3 * 4)</code>                                        | stdout: 14                                                                                | stdout: 14                                                                                | ✓<br>PASS |
| TC-30 | Python Code Execution — With Input()              | <code>name = input('Enter name: ') print('Hello', name)</code>       | Input modal appears. User enters value. Output shows Hello [name].                        | Input modal appears. User enters value. Output shows Hello [name].                        | ✓<br>PASS |
| TC-31 | Python Code Execution — Infinite Loop             | <code>while True: pass</code>                                        | Execution stopped after 5 seconds. Timeout error displayed.                               | Execution stopped after 5 seconds. Timeout error displayed.                               | ✓<br>PASS |
| TC-32 | Python Code Execution — Syntax Error              | <code>print('hello' print('world'))</code>                           | SyntaxError message shown with line number. Code not executed.                            | SyntaxError message shown with line number. Code not executed.                            | ✓<br>PASS |
| TC-33 | Python Code Execution — Runtime Error             | <code>x = 1 / 0</code>                                               | stderr: ZeroDivisionError: division by zero                                               | stderr: ZeroDivisionError: division by zero                                               | ✓<br>PASS |
| TC-34 | JavaScript Code Execution — Basic Output          | <code>console.log('Hello JS')</code>                                 | stdout: Hello JS displayed in Output panel.                                               | stdout: Hello JS displayed in Output panel.                                               | ✓<br>PASS |
| TC-35 | JavaScript Code Execution — let/const Variables   | <code>let num1 = 10; let num2 = 20; console.log(num1 + num2);</code> | stdout: 30                                                                                | stdout: 30                                                                                | ✓<br>PASS |
| TC-36 | JavaScript Code Execution — Infinite Loop Timeout | <code>while(true){}</code>                                           | Execution stopped after 5 seconds. Timeout error displayed.                               | Execution stopped after 5 seconds. Timeout error displayed.                               | ✓<br>PASS |
| TC-37 | Password SHA-256 Hashing                          | Register with password: Test@123                                     | Password stored as SHA-256 hash in DB. Plain text never stored.                           | Password stored as SHA-256 hash in DB. Plain text never stored.                           | ✓<br>PASS |

|       |                                          |                                                      |                                                                                                                                                                |                                                                                   |           |
|-------|------------------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------|
| TC-38 | Code Submission Encryption               | Submit Python code via editor                        | Source code stored as AES-256 Fernet encrypted text in CODE_SUBMISSIONS table.                                                                                 | Source code stored as AES-256 Fernet encrypted text in CODE_SUBMISSIONS table.    | ✓<br>PASS |
| TC-39 | Code Decryption on Retrieval             | View Recent Activity — click Restore                 | Encrypted code decrypted correctly. Original code restored in editor.                                                                                          | Encrypted code decrypted correctly. Original code restored in editor.             | ✓<br>PASS |
| TC-40 | Admin Route Protection — Non-Admin Token | Regular user token used on /admin/stats              | HTTP 403 error: Admin access required                                                                                                                          | HTTP 403 error: Admin access required                                             | ✓<br>PASS |
| TC-41 | Admin Route Protection — Invalid Token   | Random invalid token used on /admin/users            | HTTP 401 error: Invalid or expired session                                                                                                                     | HTTP 401 error: Invalid or expired session                                        | ✓<br>PASS |
| TC-42 | Admin Dashboard — Overview Stats         | Admin logs in. Clicks Overview tab.                  | State cards show: Total Users, Code Submissions, Flowcharts Generated, Explanations, Python Submissions, JS Submissions, New Users This Week, Total Downloads. | All 8 stat cards displayed with correct counts.                                   | ✓<br>PASS |
| TC-43 | Admin Dashboard — View Users List        | Admin clicks Users tab.                              | Table shows all non-admin users with ID, username, mobile, join date, submission count.                                                                        | Table shows all non-admin users with correct data.                                | ✓<br>PASS |
| TC-44 | Admin Dashboard — Delete User            | Admin clicks Delete on a regular user                | User removed from database. Removed from displayed list. Success toast shown.                                                                                  | User removed from database. Removed from displayed list. Success toast shown.     | ✓<br>PASS |
| TC-45 | Admin Dashboard — View Submissions       | Admin clicks Submissions tab.                        | Table shows code submissions with ID, username, language, size (chars), upload time. Source code NOT shown (encrypted).                                        | Table shows submissions metadata. Encrypted indicator shown. No raw code exposed. | ✓<br>PASS |
| TC-46 | Python Tasks List Display                | User selects Python Basics. Clicks Tasks button.     | Python beginner task list displayed with task names, timer, and hints.                                                                                         | Python task list displayed correctly.                                             | ✓<br>PASS |
| TC-47 | JavaScript Tasks List Display            | User selects JavaScript Basics. Clicks Tasks button. | JavaScript beginner task list displayed separately from Python tasks.                                                                                          | JavaScript task list displayed correctly.                                         | ✓<br>PASS |
| TC-48 | Task Timer Countdown                     | User clicks on a task. Timer starts.                 | Countdown timer runs. Shows time remaining. Turns red when under 1 minute.                                                                                     | Timer counts down correctly. Red warning shown when < 60 seconds.                 | ✓<br>PASS |
| TC-49 | Task Hint System                         | User cannot solve task. Clicks Get Hint.             | Hint text displayed explaining approach for that specific task.                                                                                                | Relevant hint displayed for the task.                                             | ✓<br>PASS |

|       |                                                  |                                                                           |                                                                                    |                                                            |           |
|-------|--------------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------|-----------|
| TC-50 | Recent Activity — Code Restore                   | User clicks Restore on a past Python submission in Recent Activity panel. | Code loaded into editor. Language set to Python. Previous result cleared.          | Code loaded correctly. Language set correctly.             | ✓<br>PASS |
| TC-51 | Dashboard Language Selection — Python Basics     | User clicks Python Basics card on Dashboard.                              | Editor page opens with Python language selected. Placeholder shows Python example. | Editor opens with Python selected correctly.               | ✓<br>PASS |
| TC-52 | Dashboard Language Selection — JavaScript Basics | User clicks JavaScript Basics card on Dashboard.                          | Editor page opens with JavaScript language selected. Placeholder shows JS example. | Editor opens with JavaScript selected correctly.           | ✓<br>PASS |
| TC-53 | Run Code Button Size and Style                   | View Run Code button in editor.                                           | Button is large (13px 28px padding), teal border, visible and prominent.           | Button displays with correct size and teal border styling. | ✓<br>PASS |
| TC-54 | Generate Flowchart Button Style                  | View Generate Flowchart button in editor.                                 | Button is large, filled teal background, glow shadow, clearly visible.             | Button displays with teal fill and glow effect.            | ✓<br>PASS |
| TC-55 | Username Placeholder in Login                    | View login form username input.                                           | Placeholder text shows: Enter your username                                        | Placeholder shows: Enter your username                     | ✓<br>PASS |
| TC-56 | Sign Out Functionality                           | User clicks Sign Out in navbar.                                           | Token cleared from localStorage. Redirected to login page.                         | Token cleared. Login page displayed.                       | ✓<br>PASS |
| TC-57 | Toast Notifications                              | Generate flowchart successfully.                                          | Green success toast shown: Flowchart ready — X nodes                               | Green toast displayed with correct message.                | ✓<br>PASS |
| TC-58 | Forgot Password Link on Login                    | User clicks Forgot password? on login form.                               | Forgot password flow starts. Step 1 (mobile entry) shown.                          | Forgot password page displayed correctly.                  | ✓<br>PASS |

## **5.2 System Implementation**

### **5.2.1 Implementation Procedures**

#### **1. Authentication Module Testing**

The registration and login modules were tested using valid and invalid credentials. Test cases verified duplicate username detection, password validation, session expiry, admin access control, and OTP-based password reset functionality.

#### **2. Flowchart Generation Testing**

The system processed Python and JavaScript code to generate flowcharts containing process, decision, loop, and function nodes. Test cases verified correct diagram generation, node alignment, drag functionality, and PNG download support.

#### **3. Code Execution Testing**

Python and JavaScript execution modules were tested for output generation, arithmetic operations, user input handling, syntax errors, runtime exceptions, and infinite loop timeout protection.

#### **4. Security Testing**

Security testing verified SHA-256 password hashing, AES-256 encrypted code storage, secure code restoration, and admin route protection using token validation.

#### **5. Admin Dashboard Testing**

The admin module was tested for viewing users, monitoring submissions, generating statistics, and deleting users securely without exposing encrypted source code.

#### **6. UI and Task Module Testing**

The user interface was tested for dashboard navigation, task timers, hints, toast notifications, button styling, language selection, and sign-out functionality.

## **CHAPTER – VI ADVANTAGES AND DISADVANTAGES**

## 6.1 Advantages and Disadvantages

### Advantages

1. **Unified Platform:** Combines code editing, execution, and flowchart visualization in a single browser-based tool — no separate applications needed.
2. **Dual Language Support:** Supports both Python Basics and JavaScript Basics, covering the two most commonly taught programming languages for beginners.
3. **Security First:** All passwords are SHA-256 hashed and all code submissions are AES-256 encrypted. No user data is stored in plain text at any point.
4. **Zero Installation:** Fully browser-based — accessible from any device with a modern browser and internet connection.
5. **Interactive Flowcharts:** React Flow-based diagrams allow users to drag, zoom, pan, and explore their flowchart dynamically.
6. **Correct PNG Export:** The html-to-image approach correctly captures all SVG edges and connection arrows in the downloaded PNG, unlike html2canvas-based solutions.
7. **Automatic Layout:** Dagre layout library ensures nodes are always properly spaced and non-overlapping when a flowchart is first generated.
8. **Admin Control:** The Admin Dashboard gives platform operators real-time visibility into user activity and account management without direct database access.
9. **Free Cloud Deployment:** Designed to run entirely on free-tier cloud services (Vercel, Railway, Supabase, Fast2SMS), making it cost-free for academic and educational use.
10. **User Trust by Design:** Multiple security layers, 24-hour session expiry, account deletion capability, and transparent data practices build verifiable user trust.

## **Disadvantages**

1. **In-Memory Sessions:** Session tokens are stored in server memory, meaning all active sessions are lost on backend restart. A production system should use Redis or a database-backed session store.
2. **Limited Language Support:** Currently restricted to Python and JavaScript. Adding Java, C++, or C# would require significant additional parser and execution engine development.
3. **Subprocess Sandboxing:** The subprocess-based sandbox provides basic isolation but is not equivalent to Docker container sandboxing. Advanced malicious code could potentially exhaust server resources.
4. **AST Accuracy:** The flowchart engine handles common constructs well but may produce incomplete diagrams for highly complex, nested, or dynamically generated code patterns.
5. **SMS OTP Dependency:** Fast2SMS offers 200 credits free SMS verifications per month. High user volumes could incur costs, and network issues could delay OTP delivery.
6. **No Offline Support:** As a web application, Ragsy requires an active internet connection. Progressive Web App (PWA) or offline mode is not currently implemented.

## CHAPTER – VII CONCLUSION

## **7.1 Conclusion**

Ragsy — Code Architecture Visualizer successfully demonstrates the feasibility and value of a unified, browser-based platform for interactive code learning and visualisation. By combining AST-based flowchart generation, live sandboxed code execution, secure multi-user authentication, AES-256 encrypted data storage, Fast2SMS SMS-based password recovery, and an Admin Dashboard — all within an intuitive and humanised user interface — the project addresses the core learning challenges faced by beginner programmers in a technically rigorous and security-conscious manner.

The platform was developed using a modern, industry-standard technology stack (React, FastAPI, SQL Server/PostgreSQL, React Flow, Dagre, Fast2SMS, Fernet) following Agile development principles across four focused sprints. All stated objectives were fully achieved: flowcharts are generated with proper non-overlapping Dagre layout, PNG downloads correctly capture all SVG edge connections, code submissions are AES-256 encrypted in the database, the forgot password flow uses Fast2SMS SMS OTP verification, the Admin Dashboard provides complete platform oversight, and JavaScript Basics is fully functional alongside Python Basics.

Ragsy stands as a technically sound, security-conscious, educationally valuable, and cost-effective software system. Its architecture supports free cloud deployment via Vercel, Railway, and Supabase — making it accessible to educational institutions and learners globally without infrastructure cost.

## 7.2 Scope for Future Enhancement

1. **Multi-Language Support:** Extend the flowchart engine and execution sandbox to support Java, C, C++, and TypeScript with language-specific AST parsers.
2. **AI-Powered Code Suggestions:** Integrate an LLM to suggest code improvements, identify logical errors, and explain flowchart paths in natural language.
3. **Collaborative Mode:** Allow multiple users to view and co-edit the same flowchart in real time using WebSocket-based state synchronisation.
4. **Docker Sandboxing:** Replace subprocess-based execution with Docker containers for stronger, production-grade security isolation of user code.
5. **Redis Session Store:** Replace in-memory session storage with Redis for persistent, scalable session management across multiple server instances.
6. **Mobile Application:** Develop a React Native mobile app for iOS and Android, enabling code learning on smartphones and tablets.
7. **Classroom Mode:** Allow teachers to create classrooms, assign coding exercises, and view students' flowchart submissions and execution results.
8. **PWA Support:** Enable Progressive Web App capabilities so Ragsy can be installed and used with cached content on low-connectivity networks.
9. **Code History Diff:** Visually compare flowcharts of two versions of the same code to highlight structural changes.

## ANNEXURE – B

## BIBLIOGRAPHY

1. Python Software Foundation. (2024). ast — Abstract Syntax Trees. Python 3.12 Standard Library Documentation.  
<https://docs.python.org/3/library/ast.html>
2. Ramirez, S. (2023). FastAPI Documentation — Python Web Framework. <https://fastapi.tiangolo.com>
3. xyflow Team. (2024). React Flow Documentation — Interactive Node-Based UI. <https://reactflow.dev>
4. Dagre Contributors. (2023). Dagre — Directed Graph Layout for JavaScript. GitHub Repository. <https://github.com/dagrejs/dagre>
5. bubkoo. (2023). html-to-image — Generates image from a DOM node using HTML5 canvas. GitHub.  
<https://github.com/bubkoo/html-to-image>
6. Google LLC. (2024). Fast2SMS Authentication — Phone Number Sign-In Documentation.  
<https://Fast2SMS.google.com/docs/auth/web/phone-auth>
7. Fast2SMS Admin SDK. (2024). Verify ID Tokens — Fast2SMS Admin Python SDK.  
<https://Fast2SMS.google.com/docs/auth/admin/verify-id-tokens>
8. Python Cryptography Authority. (2024). Fernet Symmetric Encryption — cryptography library.  
<https://cryptography.io/en/latest/fernet>

9. OWASP Foundation. (2023). Password Storage Cheat Sheet.  
[https://cheatsheetseries.owasp.org/cheatsheets/Password\\_Storage\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)
10. Supabase Inc. (2024). Supabase Documentation — Open Source Fast2SMS Alternative with PostgreSQL. <https://supabase.com/docs>
11. Vercel Inc. (2024). Vercel Platform Documentation — Frontend Cloud Deployment. <https://vercel.com/docs>
12. Railway Technologies Inc. (2024). Railway Documentation — Infrastructure Platform. <https://docs.railway.app>
13. Microsoft Corporation. (2023). SQL Server 2019 Documentation. <https://docs.microsoft.com/en-us/sql/sql-server>
14. Meta Open Source. (2024). React — The Library for Web and Native User Interfaces. <https://react.dev> Evan You & Vite Contributors. (2024). Vite — Next Generation Frontend Tooling.